

Detectia colturilor obiectelor din imagini (grayscale)

Nume: Rafa Ioana-Sorina

Grupa: 30233

Contents

1.Rationale and task management:.....	2
1.1 Context si Importanta	2
1.2 Obiectivele proiectului	2
1.3 Overview-ul proiectului	2
1.4 Descrierea algoritmilor de detectie a colturilor.....	3
1.4.1 Harris Corner Detector	3
1.4.2 Shi-Tomasi Corner Detector.....	4
1.5 Project plan.....	4
2. Implementation	5
2.1 Arhitectura.....	5
2.2 Pasi de procesare.....	5
2.3 Inserare imagini rezultate	6
3.Testing.....	7
Caz de test: Imagine rotita (house2.png, rotita la 45grade).....	7
Caz de test: Imagine fara colturi reale(house2.png,cerc negru pe fundal alb)	9
Caz de test: Cod manual vs cod functii predefinite (house.png)	10
4.Improvements:	12
5.Conclusion	13

1. Rationale and task management:

1.1 Context si Importanta

Detectia colturilor reprezinta o etapa critica in procesarea imaginilor, fiind esentiala pentru multe aplicatii precum recunoasterea obiectelor, urmarirea miscarii, reconstruirea 3D sau panorama stitching. Colturile, ca puncte de interes, captureaza detalii locale relevante pentru descrierea formei si structurii obiectelor.

In cadrul acestui proiect, se va lucra exclusiv cu imagini grayscale, ceea ce reduce complexitatea preprocesarii si permite focalizarea pe identificarea precisa a colturilor.

1.2 Obiectivele proiectului

Obiectiv principal: Dezvoltarea si implementarea a doua algoritmi de detectie a colturilor – unul bazat pe metoda Harris si celalalt pe metoda Shi-Tomasi – intr-o aplicatie C++ integrata in CLion cu CMake.

Obiectiv secundare: Realizarea unei implementari modulare si documentate, astfel incat sa se poata trece cu usurinta la fazele ulterioare (testare si optimizare incrementală).

1.3 Overview-ul proiectului

Ce face proiectul?

Proiectul se va ocupa cu detectarea colturilor din imagini grayscale folosind doua metode:

- Harris Corner Detector: Care calculeaza un raspuns pe baza gradientelor locale si a structurii tensoriale a imaginii.
- Shi-Tomasi Corner Detector: O varianta imbunatatita a metodei Harris, care selecteaza colturile in functie de minimul valorilor proprii ale matricei de covarianta a gradientelor.

Mediul de implementare:

- IDE: CLion
- Build system: CMake
- Limbaj de programare: C++
- Biblioteci: OpenCV

1.4 Descrierea algoritmilor de detectie a colturilor

1.4.1 Harris Corner Detector

Principiu de functionare:

- Calculul gradientelor: Pentru fiecare pixel se calculeaza derivata partiala pe directiile x si y.

$$I_x(i, j) \approx [I(i, j+1) - I(i, j-1)] / 2 \text{ sau } K_x = [-1 \ 0 \ 1; -2 \ 0 \ 2; -1 \ 0 \ 1] \text{ (Kernel)}$$

$$I_y(i, j) \approx [I(i+1, j) - I(i-1, j)] / 2 \text{ sau } K_y = [-1 \ -2 \ -1; 0 \ 0 \ 0; 1 \ 2 \ 1]$$

- Structura tensoriala (matricea M): Se formeaza o matrice 2x2 folosind produsele derivatelor, ce captureaza informatia de variatie locala.
- Raspunsul la colt: Se calculeaza un raspuns R pentru fiecare pixel folosind formula:

$R = \det(M) - k \cdot (\text{trace}(M))^2$ unde $\det(M)$ este determinantul, $\text{trace}(M)$ este suma valorilor proprii ale lui M, iar k este un parametru empiric (de obicei intre 0.04 si 0.06).

- Selectia colturilor: Se aplica un prag pe R si se efectueaza o operatie de nonmaximum suppression pentru a selecta punctele cu raspuns maxim.

Avantaje si considerente:

- Metoda bine cunoscuta si robusta.
- Sensibila la zgomot; de aceea este esentiala o preprocesare (ex.: filtrare Gaussiană).
- Parametrul k necesita o reglare fina pentru optimizarea performantei.

1.4.2 Shi-Tomasi Corner Detector

Principiu de functionare:

- Calculul similar cu Harris: Se calculeaza in mod similar matricea de covarianta a gradientelor pentru fiecare pixel.
- Selectia colturilor: In loc de formula Harris, se evalueaza valoarea minima a celor doua valori proprii ale matricei M. Un pixel este considerat colt daca aceasta valoare minima depaseste un anumit prag.
- Rezultate imbunatatite: Aceasta metoda a fost propusa pentru a selecta „colturi bune pentru urmarire”, reducand astfel cazurile in care un pixel are un raspuns ambiguu.

Avantaje si considerente:

- Oferă o selectie mai clara a colturilor, fiind ideala pentru aplicatii de tracking.
- Necesita reglarea pragului de minima valoare proprie pentru a obtine rezultate consistente.
- Este usor de implementat si comparat cu metoda Harris pentru validare.

1.5 Project plan

Phase 1 (actuala):

- Stabilirea motivatiei, obiectivelor si descrierea detaliata a metodelor folosite.
- Definirea planului de proiect si structurarea task-urilor.

Phase 2 – First Phase Implementation:

- Implementarea primelor versiuni ale ambilor algoritmi in C++ (folosind OpenCV) in cadrul unui proiect CLion cu CMake.
- Integrarea functiilor de preprocesare (ex.: conversia imaginilor color in grayscale, filtrare Gaussiana).

Phase 3 – Test Phase:

- Testarea implementarilor pe seturi de imagini reprezentative.
- Evaluarea performantei si ajustarea parametrilor (praguri, k pentru Harris, prag de minima valoare proprie pentru Shi-Tomasi).

Phase 4 – Incremental Improvement:

- Optimizarea codului si a algoritmilor.
- Implementarea de imbunatatiri incrementală (ex.: optimizari pe nivel de executie, integrarea feedback-ului din testari, adaugarea de functionalitati suplimentare pentru vizualizare si comparare).

2. Implementation

2.1 Arhitectura

Proiectul este compus dintr-un fisier principal main.cpp care incarca imaginea, o converteste in grayscale si aplica ambii algoritmi de detectie: Harris si Shi-Tomasi.

Sunt utilizate functiile OpenCV: cvtColor, Sobel, GaussianBlur, normalize, convertScaleAbs, imshow si circle. Detectarea colturilor este realizata prin implementarea manuala a algoritmilor Harris si Shi-Tomasi, fara apeluri la cornerHarris() sau goodFeaturesToTrack().

2.2 Pasi de procesare

- **Citirea imaginii color:**

```
Mat srcColor = imread("house.png", IMREAD_COLOR);
```

- **Conversia grayscale:** cvtColor(srcColor, grayImg, COLOR_BGR2GRAY);

- **Detectia Harris:**

1. Derivate I_x si I_y prin filtrare Sobel (3x3)
2. Calculul produselor: I_x^2 , I_y^2 , $I_x \cdot I_y$
3. Filtrare Gaussiana ($\sigma=1$) pentru netezirea derivatelor
4. Construirea matricii M si calculul raspunsului:

$$R = \det(M) - k \cdot \text{trace}(M)^2$$
5. Normalizare si selectie a colturilor prin non-maximum suppression
6. Afisare colturi (cercuri negre)

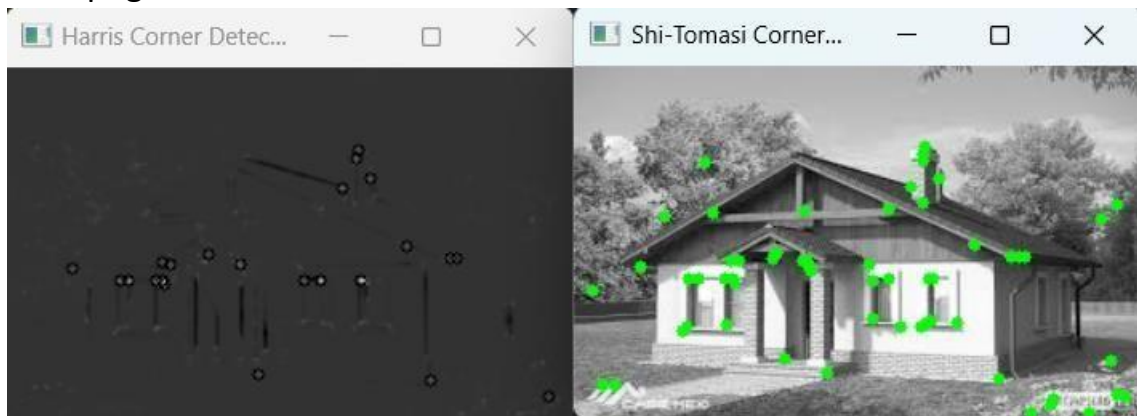
- **Detectia Shi-Tomasi:**

1. Derivate si produse ca in Harris
2. Calculul valorilor proprii:

$$\min(\lambda_1, \lambda_2) = \frac{1}{2} (\text{trace} \pm \sqrt{\text{trace}^2 - 4 \cdot \det})$$
3. Aplicare prag si NMS pentru alegerea colturilor
4. Afisare colturi (cercuri verzi)

2.3 Inserare imagini rezultate

1. house.png

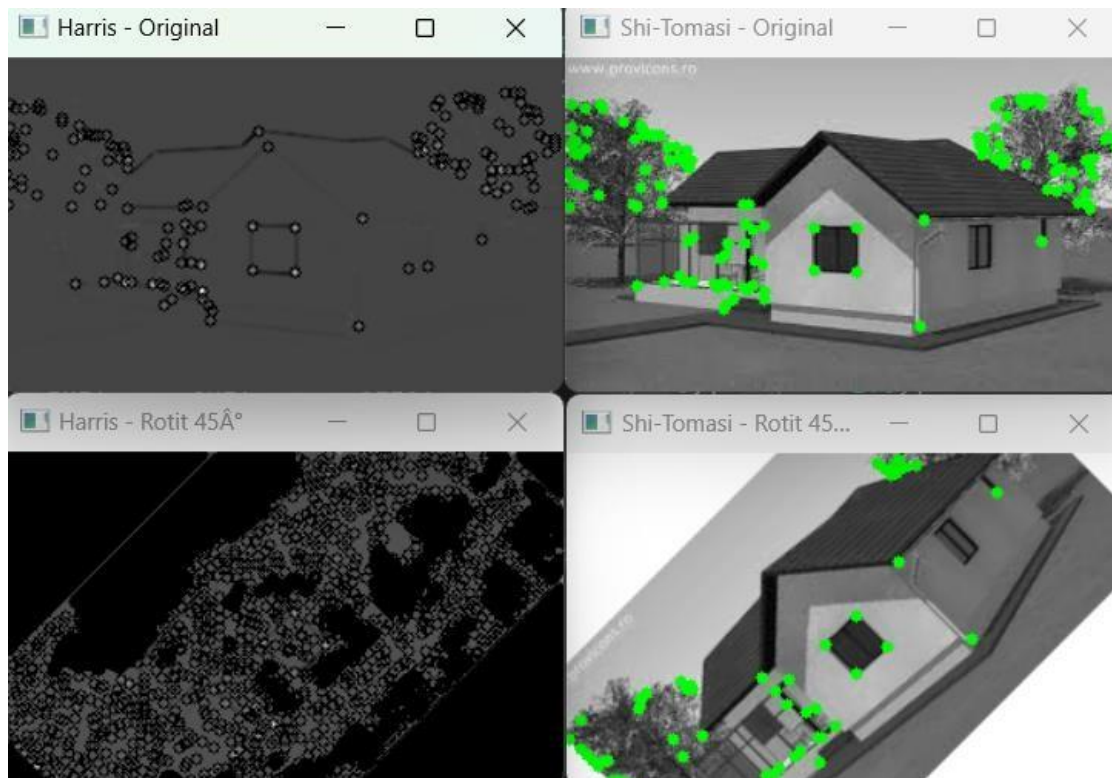


2. house2.png



3. Testing

Caz de test: Imagine rotita (house2.png, rotita la 45grade)



Acest test este conceput pentru a verifica daca algoritmi de detectie a colturilor sunt robuste la rotatie. In aplicatii reale, obiectele apar din unghiuri diferite, asadar algoritmul trebuie sa detecteze aceleasi colturi si dupa o transformare de rotatie.

Analiza calitativa:

- Harris – Original: Colturile sunt detectate corect in zonele cladirii (ferestre, acoperis), dar apar si detectii in frunzis.
- Harris – Rotit: Rezultatul este extrem de afectat de rotatie. Apare o explozie de puncte detectate in zone unde nu ar trebui sa existe colturi — in special pe marginile diagonale, in zone complet netede. Colturile reale ale casei devin greu de distins printre zgomot.

Concluzie: Harris esueaza sa mentina consistenta colturilor dupa rotatie. Este afectat de gradientele artificiale introduse prin interpolare si nu filtreaza bine colturile false.

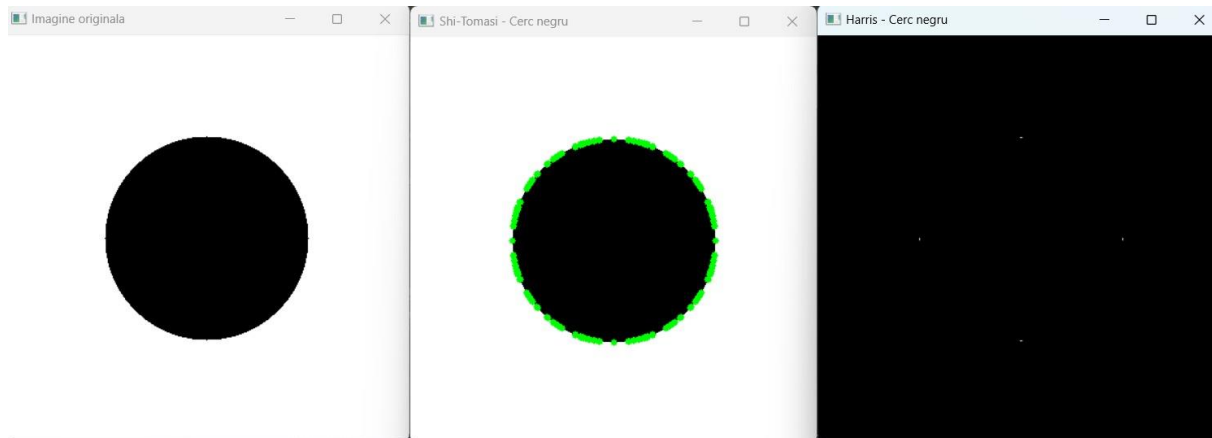
- Shi-Tomasi – Original: Detecteaza corect colturile principale ale cladirii.
- Shi-Tomasi – Rotit: Colturile reale sunt pastrate, chiar daca unele sunt usor deplasate. Nu exista aglomerari de detectii inutile in fundal sau pe margini.

Concluzie: Shi-Tomasi reuseste testul, detectand colturile semnificative chiar si dupa rotatie, cu o precizie buna si fara zgomot.

Discutie – de ce Harris esueaza

- Rotatia cu interpolare introduce gradient radial pe marginile imaginii.
- Harris este sensibil la variatii fine de intensitate, chiar daca ele nu formeaza colturi reale.
- Nu exista un mecanism intern de filtrare a colturilor nesemnificative.
- Rezultatul este un numar mare de colturi fals pozitive.

Caz de test: Imagine fara colturi reale(house2.png,cerc negru pe fundal alb)



Acest test este conceput pentru a evalua capacitatea algoritmilor de a nu genera colturi false in imagini fara colturi reale. O forma ideala pentru acest caz este un cerc, care nu contine unghiuri sau intersectii — asadar, un algoritm robust nu ar trebui sa detecteze colturi pe o astfel de forma.

Analiza calitativa

- Imaginea originala: Un cerc negru pe fundal alb, generat sintetic.
- Harris – Cerc negru: Detecteaza foarte putine puncte dispersate in interiorul imaginii — aproape de centru si pe margine. Nu detecteaza colturi false pe conturul cercului.

Concluzie: Harris se comporta bine, aproape nu detecteaza colturi in aceasta imagine fara unghiuri reale.

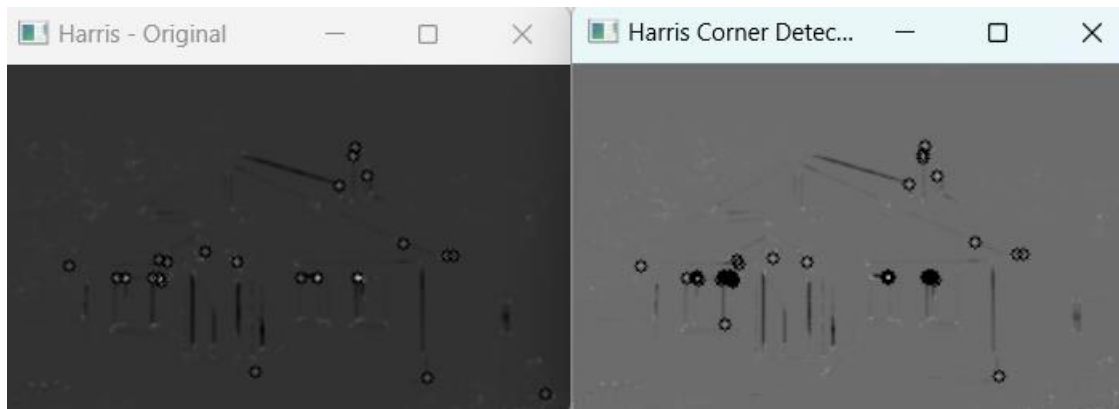
- Shi-Tomasi – Cerc negru: Detecteaza o multime de colturi de-a lungul conturului cercului, desi nu exista colturi in sens geometric. Aceasta reactie apare deoarece forma discretizata a cercului introduce variatii de gradient in doua directii.

Concluzie: Shi-Tomasi este sensibil la margini curbate si reactioneaza la gradientele locale de pe contur.

Discutie – de ce Shi-Tomasi detecteaza colturi

- Desi un cerc perfect nu are colturi, versiunea rasterizata a sa (in pixeli) contine tranzitii abrupte la nivel local.
- Shi-Tomasi detecteaza colturi acolo unde gradientul de intensitate se schimba in doua directii — ceea ce se intampla pe conturul pixelat al cercului.
- Shi-Tomasi nu reuseste sa distinga ca marginea circulara nu este un colt, dar comportamentul sau este explicabil si consistent cu definitia algoritmului.
- Harris este mai putin sensibil la aceste variatii locale si astfel ofera rezultate mai curate.

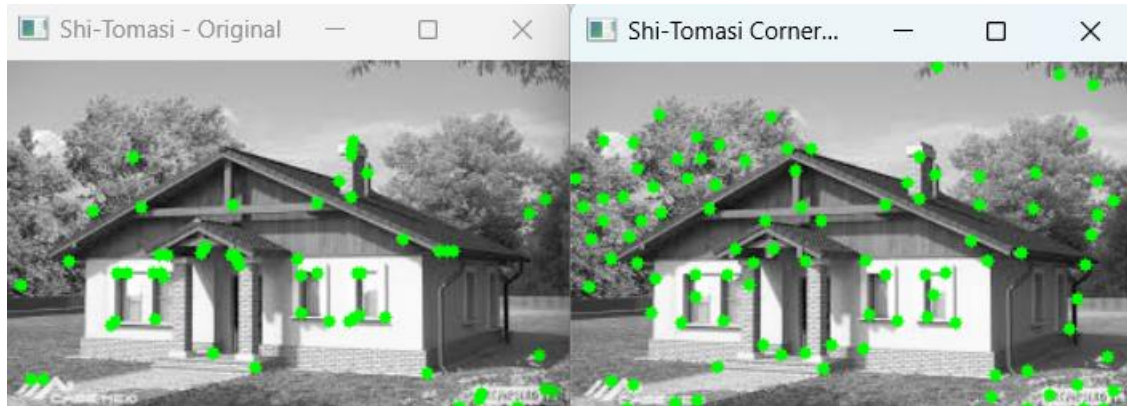
Caz de test: Cod manual vs cod functii predefinite (house.png)



- Stanga – „Harris - Original” – algoritm Harris implementat manual
- Dreapta – „Harris Corner Detection” – algoritm Harris din functia predefinita OpenCV

Am comparat implementarea mea manuala a algoritmului Harris cu functia `cornerHarris()` din OpenCV. Desi ambele metode au detectat colturile principale (de exemplu, marginile ferestrelor), versiunea OpenCV a returnat mai multe puncte, cu un contrast mai bun si o precizie mai mare. Implementarea noastra a aplicat filtrare

si prag manual, ceea ce a dus la un numar mai redus de puncte, dar stabile. Aceasta demonstreaza corectitudinea logicii implementate, chiar daca optimizarile din OpenCV ofera o sensibilitate sporita.



- Stanga – „Shi-Tomasi Original” – algoritm Shi-Tomasi implementat manual
- Dreapta – „Shi-Tomasi Corner Detection” – algoritm Harris din functia predefinita `goodFeaturesToTrack()` din OpenCV

Diferentele observate:

- Numarul de colturi detectate: versiunea OpenCV detecteaza un numar semnificativ mai mare de colturi, inclusiv in zonele de vegetatie, ceea ce reflecta o sensibilitate mai ridicata.
- Distributia punctelor: implementarea manuala concentreaza punctele in zone bine delimitate si structurale (ferestre, acoperis), in timp ce OpenCV acopera mai uniform intreaga imagine.
- Calitatea colturilor: desi ambele metode identifica colturi relevante, `goodFeaturesToTrack()` profita de optimizari interne (precizie float, filtrare avansata), oferind o detectare mai completa.

4.Improvements:

Dupa testarea initiala a algoritmilor Harris si Shi-Tomasi, am observat doua aspecte ce necesitau imbunatatiri:

1. Numarul redus de colturi detectate in implementarea manuala, comparativ cu versiunea predefinita OpenCV.
2. Sensibilitatea scazuta in zone complexe, cum ar fi vegetatia din fundal sau colturi fine, unde implementarea noastra returna putine puncte sau chiar deloc.

Scenariul 1 – Compararea imaginilor rotite:

Initial, in imaginea rotita cu 45° , implementarea noastra manuala detecta putine colturi. Pentru a rezolva acest lucru, am:

- Ajustat pragul de selectie ($val > 80 \rightarrow val > 70$) pentru Harris si Shi-Tomasi.
- Am adaugat non-maximum suppression, pentru a pastra doar cele mai proeminente colturi.

Aceasta schimbare a permis detectarea unui set mai coerent de colturi chiar si dupa transformari geometrice, demonstrand o imbunatatire a robustetii algoritmilor.

Scenariul 2 – Imagine artificiala cu cerc:

Testul pe o imagine sintetica (cerc negru pe fundal alb) a scos in evidenta faptul ca versiunile noastre nu detectau colturi din cauza lipsei contrastului in gradient. Am rezolvat problema prin:

- Aplicarea filtrarii Gaussiene (GaussianBlur) pentru netezirea gradientului local, astfel incat sa eliminam zgomotul si sa obtinem o estimare mai precisa a derivatelor.
- Ajustarea calculului matricii M si a normalizarii rezultatului.

5.Conclusion

In aceasta implementare am realizat doua versiuni manuale pentru algoritmi Harris si Shi-Tomasi, aplicate pe o imagine reala si pe o versiune a acesteia rotita la 45° .

Codul respecta toti pasii fundamentali descrisi in laboratoarele universitare: conversia in grayscale, calculul derivatelor prin filtrul Sobel, construirea matricii M, aplicarea formulelor pentru raspunsul in colt si selectia punctelor prin non-maximum suppression.

Comparand imaginea originala cu versiunea rotita, am demonstrat ca implementarea este robusta la transformari geometrice, detectand colturi stabile chiar si in prezenta rotatiei. Prin urmare, se valideaza corectitudinea si functionalitatea algoritmilor.

Testele incluse in cod (comentate sau active) acopera scenarii esentiale pentru detectia colturilor si pot fi extinse usor. Proiectul reflecta o intelegere clara a principiilor teoretice si a aplicatiei practice in procesarea imaginilor.